Registro de fuentes y uso responsable de IA

Adjunta este registro a tus tareas cuando utilices herramientas de IA.

Actividad / Código: Practica de desarrollo web Estudiante: Cristian Ucsiña

Declaración de uso:

- Utilicé IA generativa como apoyo. Documentaré a continuación los prompts relevantes y verificaré las referencias antes de citarlas.

Registra una fila por consulta significativa (puedes duplicar la tabla).

Herramienta / modelo	Gemini
Fecha	12/11/2026
Propósito de la consulta	como realizo el control de estado global y la sincronizaacion de la interfaz
Prompt exacto	Al momento de realizar el index no entendi como realizar un control de estado por que no se me sincroniza con cliente, administrador y el inactivo de la cuenta. Ayudame a entender como funciona y como lo puedo implementar en mi tarea
Respuesta aprovechada	Entiendo perfectamente por qué esta parte suele causar dolores de cabeza. En una página web tradicional, cada pantalla es un archivo diferente y el servidor se encarga de la seguridad. Pero en tu aplicación de una sola página (**SPA**), **todo vive en el mismo `index.html`**, lo que significa que **JavaScript debe actuar como el "portero"** que decide qué se muestra, qué se oculta y quién puede entrar. Para que el estado se sincronice correctamente entre **clientes, administradores y cuentas inactivas**, necesitas controlar tres elementos clave: **el almacenamiento del usuario actual, la validación del login (bloqueando a los inactivos) y la actualización visual del menú**. **¿Cómo funciona el control de estado? (Paso a paso)** 1. **El Estado (`usuarioActual`):** Es una variable global en tu `script.js` (o guardada en `localStorage`) que responde a la pregunta: *¿Quién está usando la aplicación en este momento?* Si es `null`, es un **invitado**. Si tiene datos, guardará su rol (`cliente` o `admin`) y su estado (`activo` o `inactivo`).

	2. **El Login y las cuentas inactivas:** Cuando el usuario presiona "Entrar", el sistema busca en la lista de usuarios. Si las credenciales coinciden pero el estado del usuario es **inactivo** o está bloqueado por intentos fallidos, el sistema **debe denegar el acceso** con una alerta y evitar cambiar el estado. 3. **La Sincronización Visual (El Menú):** Cada vez que alguien inicia o cierra sesión, ejecutamos una función que recorre las clases de tu menú (`.nav-invitado`, `.nav-admin`, `.nav-solo-cliente`) para mostrarlas (`display: block` o removiendo la clase `.oculto`) o esconderlas según corresponda.
Fuentes informales y sugeridas por IA (al menos 3)	▪ **Diseño de interfaz y maquetación (Bootstrap):** Asistencia en la estructura de la cuadrícula responsiva, diseño de tarjetas de servicios y la sección "Acerca de" con estética oscura personalizada mediante CSS. ▪ **Lógica de autenticación y roles (JavaScript):** Orientación en la configuración del script de inicio de sesión simulado, manejo de arreglos para cuentas de prueba y validación de campos de formulario. ▪ **Optimización y depuración de componentes:** Apoyo en la revisión de errores de visibilidad en modales, corrección de selectores CSS y estructuración general de archivos del proyecto (`css/`, `js/`, carpetas de imágenes).
Fuentes verificadas y citadas (al menos 3)	• **Gestión de estado y arquitectura de componentes:** El artículo *"Best Practices for Building Scalable Single Page Applications (SPAs)"* analiza aspectos críticos en el desarrollo web moderno, incluyendo la arquitectura de componentes y las estrategias de gestión de estado. • **Estilo arquitectónico escalable:** El estudio *"An Architectural Style for Single Page Scalable Modern Web"* describe modelos y patrones de diseño adecuados para estructurar interfaces complejas en el lado del cliente. • **Estudios de caso en rendimiento:** La investigación *"Modern Single Page Application Architecture: A Case Study"* examina metodologías para lograr altos niveles de rendimiento y experiencias de usuario centradas en componentes.

Bibliografía / referencias verificadas

Elliott, E. (s. f.). *Programming JavaScript Applications: Robust Web Architecture with Node, HTML5, and Modern JS Libraries*. O'Reilly Media.

Mozilla Contributors. (s. f.). *Single-page application (SPA) architecture & Client-side routing*. Mozilla Developer Network (MDN).

Rossi, G., Pastor, O., Schwabe, D., y Olsina, L. (s. f.). *Web Engineering: Modelling and Implementing Web Applications*. Springer Science & Business Media.